

lab2物理内存和页表

实验目的

- 理解页表的建立和使用方法
- 理解物理内存的管理方法
- 理解页面分配算法

实验内容

实验一过后大家做出来了一个可以启动的系统，实验二主要涉及操作系统的物理内存管理。操作系统为了使用内存，还需高效地管理内存资源。本次实验我们会了解如何发现系统中的物理内存，然后学习如何建立对物理内存的初步管理，即了解连续物理内存管理，最后掌握页表相关的操作，即如何建立页表来实现虚拟内存到物理内存之间的映射，帮助我们对段页式内存管理机制有一个比较全面的了解。本次的实验主要是在实验一的基础上完成物理内存管理，并建立一个最简单的页表映射。

练习

对实验报告的要求：

- 基于markdown格式来完成，以文本方式为主
- 填写各个基本练习中要求完成的报告内容
- 完成实验后，请分析ucore_lab中提供的参考答案，并请在实验报告中说明你的实现与参考答案的区别
- 列出你认为本实验中重要的知识点，以及与对应的OS原理中的知识点，并简要说明你对二者的含义，关系，差异等方面的理解（也可能出现实验中的知识点没有对应的原理知识点）

本次实验的重要知识点如下：

内存分配：内存分配是管理物理内存的过程，包括为进程分配内存块、跟踪哪些内存块已分配和哪些是可用的。包括：

连续内存分配：将物理内存分为连续的块，适用于一些简单的系统。

分页内存分配：将内存划分成固定大小的页面，更灵活地分配内存。

内存回收：内存回收是将不再需要的内存块释放回系统以供其他进程使用的过程。知识点包括：

释放策略：何时释放内存块，如何处理进程退出时的内存回收。

碎片管理：内存分配和回收可能导致内存碎片的问题，其中有两种类型：

外部碎片：未分配但不可用的内存块散布在系统中。

内部碎片：已分配但未完全利用的内存块。

地址转换：物理内存管理还涉及将进程的逻辑地址（虚拟地址）映射到物理内存地址的过程。包括：

页表：用于虚拟地址到物理地址的映射。

页表项：页表中的每个条目，包括页帧号和权限位。

地址空间：每个进程都有自己的地址空间，包括代码段、数据段和堆栈段。

内存保护：内存管理还包括保护内存不被非授权访问的机制，通常通过页表中的权限位来实现。

页面置换算法：当物理内存不足时，操作系统需要选择哪些页面从内存中交换到磁盘以腾出空间。包括FIFO、LRU、Clock 等页面置换算法。

多进程管理：多个进程共享有限的物理内存，需要考虑如何公平地分配内存和避免进程之间的冲突。

地址空间布局：操作系统定义了每个进程的地址空间布局，通常包括代码段、数据段、堆和栈。

伙伴系统：伙伴分配算法是用于管理分配和回收内存块的方法，通过将内存分成大小相等的伙伴块来提高效率。

大页表：为了加速地址转换，一些系统使用大页表，将多个页合并为一个更大的页

- 列出你认为OS原理中很重要，但在实验中没有对应上的知识点

进程调度算法：操作系统原理通常详细研究了各种进程调度算法，例如先来先服务（FCFS）、最短作业优先（SJF）、优先级调度、轮转调度（Round Robin）等。这些算法影响了进程如何被分配处理器时间片，但在物理内存管理实验中，通常集中在内存分配和回收。

多线程管理：操作系统原理涵盖了多线程的管理和同步问题，包括线程创建、销毁、同步（例如互斥锁、条件变量）和调度。物理内存管理实验可能更侧重于单个进程的内存管理。

虚拟内存：虚拟内存管理是操作系统原理中的重要主题，涉及到页面置换、分页/分段机制、地址翻译等。虽然实验中可能会讨论内存分配，但虚拟内存通常是一个更广泛的主题。

文件系统类型：操作系统原理涵盖了不同文件系统类型的设计和实现，如FAT、NTFS、EXT等。实验可能会包括基本的文件操作，但不涵盖文件系统设计。

中断处理：操作系统原理中会详细介绍中断和异常处理机制，包括中断向量表、异常类型、中断控制器等。实验可能会处理基本的时钟中断，但不会深入研究各种中断类型和中断向量表。

进程通信和同步：操作系统原理包括进程间通信机制，如消息队列、信号量、管程等。物理内存管理实验通常集中在内存资源分配，而不深入研究进程间通信和同步。

练习1：理解first-fit 连续物理内存分配算法（思考题）

first-fit 连续物理内存分配算法作为物理内存分配一个很基础的方法，需要同学们理解它的实现过程。请大家仔细阅读实验手册的教程并结合`kern/mm/default_pmm.c`中的相关代码，认真分析`default_init`，`default_init_memmap`，`default_alloc_pages`，`default_free_pages`等相关函数，并描述程序在进行物理内存分配的过程以及各个函数的作用。

`default_init`: 这个函数的作用是在物理内存分配过程中初始化一个空闲内存块链表和相应的计数器。实现过程：

`list_init(&free_list)`: 这一行代码调用了 `list_init` 函数，它的目的是初始化一个链表结构。在这里，`free_list` 是一个链表，它用来跟踪可用的物理内存块（或者说是空闲的内存块）。通过初始化 `free_list`，确保了链表是空的，没有任何内存块加入。

`nr_free = 0`: 这一行代码将 `nr_free` 变量初始化为 0。`nr_free` 是一个用来记录空闲内存块数量的计数器。通过将其初始化为 0，表示初始时没有任何内存块是空闲的。

在物理内存分配的过程中，这个函数的作用是为系统初始化一个空闲内存块链表，并记录当前可用的内存块数量。随着物理内存的分配和释放，`free_list` 链表会动态地更新，`nr_free` 计数器也会相应地增加或减少以反映系统中的空闲内存块数量。这有助于操作系统在内存分配请求时找到适当的内存块以分配给进程。

default_init_memmap： 这个函数的作用是在物理内存分配过程中初始化一个内存页映射表（`mem_map`）和一个链表（`free_list`）。实现过程：

```
assert(n > 0) : 这行代码是一个断言，确保传递给函数的页数 n 大于 0，因为没有必要初始化一个空的映射表。

struct Page *p = base; : 创建一个指向 base 的指针 p，它将用于遍历页表中的所有页。

for (; p != base + n; p++) : 这是一个循环，从 base 开始遍历 n 个页的区域。

assert(PageReserved(p)) : 这行代码使用断言检查页 p 是否被保留。在内存初始化过程中，通常某些页面会被保留用于特定目的，例如内核代码或者设备驱动程序，这些页不应该用于通用内存分配。

p->flags = p->property = 0; : 将页的标志 (flags) 和属性 (property) 设置为零，表示这些页当前没有特殊的标志或属性。

set_page_ref(p, 0) : 将页的引用计数设置为零。引用计数是用来跟踪页的引用次数的，这里将其初始化为零，表示这些页当前没有被引用。

base->property = n; : 将 base 页的属性字段设置为 n，表示该页表映射了多少个页。

SetPageProperty(base); : 设置 base 页的一个特殊标志，表示这个页是页表。

nr_free += n; : 将系统中的空闲页数增加 n，因为在初始化过程中，这些页是空闲的。

接下来的代码段涉及将初始化的页添加到 free_list 链表中。如果 free_list 为空，直接将初始化的页添加为链表的第一个元素。如果 free_list 不为空，它会遍历链表并将初始化的页插入到适当的位置，以保持链表的有序性。
```

在物理内存分配的过程中，这个函数的作用是初始化一个内存页映射表和空闲页链表。映射表用于跟踪每个物理页的属性和状态，而空闲页链表用于维护当前可用的物理页，确保物理内存可以正确地分配和回收。

default_alloc_pages： 这个函数的作用是在物理内存分配过程中，尝试分配连续的 n 个物理内存页。实现过程如下：

```
assert(n > 0) : 这是一个断言，确保请求的页数 n 大于 0，因为没有必要分配零个或负数个页。

if (n > nr_free) : 这个条件检查系统中是否有足够的空闲页来满足请求。nr_free 记录了当前可用的空闲页数量，如果请求的页数 n 大于可用的页数，函数返回 NULL，表示无法满足分配请求。

struct Page *page = NULL; : 初始化一个指向 Page 结构的指针 page，用于记录已分配的起始页。

list_entry_t *le = &free_list; : 初始化一个链表元素指针 le，指向空闲页链表的头部。

while ((le = list_next(le)) != &free_list) : 这是一个循环，用于遍历空闲页链表。
```

`struct Page *p = le2page(le, page_link);` : 将链表元素 `le` 转换为 `Page` 结构，从而可以访问每个空闲页的属性。

`if (p->property >= n) :` 这个条件检查当前空闲页 `p` 是否有足够多的连续页来满足请求。如果是，将当前页 `p` 分配给 `page` 变量，并退出循环。

`if (page != NULL) :` 如果找到了足够的连续页来满足请求，执行以下步骤：

`list_entry_t* prev = list_prev(&(page->page_link));` : 找到 `page` 的前一个链表元素，以便稍后在链表中进行插入操作。

`list_del(&(page->page_link));` : 从空闲页链表中删除已分配的页。

`if (page->property > n) :` 如果已分配的页比请求的页数多，执行以下步骤：

`struct Page *p = page + n;` : 计算剩余连续空闲页的起始地址。

`p->property = page->property - n;` : 更新剩余空闲页的属性字段，表示新的剩余页数。

`SetPageProperty(p);` : 将新的剩余页设置为页表。

`list_add(prev, &(p->page_link));` : 将剩余空闲页添加到链表中，以维护链表的有序性。

`nr_free -= n;` : 减少系统中可用的空闲页数量，以反映已分配的页。

`ClearPageProperty(page);` : 清除已分配页的页表属性，因为它们不再是页表。

返回已分配的起始页 `page`，或者如果无法满足请求，返回 `NULL`。

这个函数的主要作用是根据请求的页数分配连续的物理内存页，并更新内存管理数据结构以反映已分配的页和剩余的空闲页，为进程分配内存块。

`default_free_pages`：这个函数的作用是在物理内存分配过程中释放一段连续的物理内存页，将这些页重新加入到可用的物理内存页链表中。实现过程如下：

`assert(n > 0) :` 这是一个断言，确保释放的页数 `n` 大于 0，因为没有必要释放零个或负数个页。

`struct Page *p = base;` : 创建一个指向 `base` 的指针 `p`，用于遍历要释放的页。

`for (; p != base + n; p++) :` 这是一个循环，从 `base` 开始遍历 `n` 个页的区域。

`assert(!PageReserved(p) && !PageProperty(p)) :` 这个断言确保被释放的每个页既不是保留页（例如内核页），也不是页表页。这样可以避免错误释放重要的页。

`p->flags = 0;` : 将页的标志字段设置为零，表示这些页当前没有特殊标志。

`set_page_ref(p, 0) :` 将页的引用计数设置为零，表示这些页不再被引用。

`base->property = n;` : 将 `base` 页的属性字段设置为 `n`，表示这个连续页块包含了多少个页。

`SetPageProperty(base);` : 设置 `base` 页的一个特殊标志，表示这个页块是页表。

`nr_free += n;` : 增加系统中的可用空闲页数，以反映已释放的页。

下面的代码段涉及将释放的页添加回空闲页链表 `free_list` 中，保持链表的有序性。如果 `free_list` 是空的，直接将释放的页添加为链表的第一个元素。如果 `free_list` 不为空，它会遍历链表并将释放的页插入到适当的位置。

接下来的代码段检查释放的页块与链表中相邻的页块是否可以合并。如果被释放的页块与链表中前一个页块相邻（连续），它们会合并成一个更大的页块，然后从链表中删除一个页块。如果被释放的页块与链表中后一个页块相邻，它们也会合并成一个更大的页块，并删除后一个页块。

这个函数的主要作用是释放一段连续的物理内存页，并更新内存管理数据结构以反映已释放的页和可用的空闲页，以便将已使用的内存返回给系统以供将来的分配。

请在实验报告中简要说明你的设计实现过程。请回答如下问题：

- 你的first fit算法是否有进一步的改进空间？

First Fit算法在处理内存碎片和提高分配效率方面存在改进空间：

内存碎片优化：一种改进First Fit的方法是采用更高级的内存分配算法，如Best Fit或Next Fit，以减少内存碎片。这些算法更有效地利用内存空间，减少了碎片的发生。

动态分区大小：First Fit通常使用静态分区大小，即将内存分成固定大小的块。一种改进是使用动态分区大小，根据不同的内存分配请求自动分配不同大小的内存块，以最大程度减少碎片。

碎片整理：实现一种碎片整理机制，周期性地整理内存以合并碎片，从而提高内存的利用率。这种方法可以通过移动数据来合并不相邻的内存块，减少碎片。

优化搜索：First Fit的一个问题是它从内存空闲块链表的起始位置开始搜索，可能需要遍历整个链表才能找到合适的块。通过使用更高效的搜索算法，可以减少搜索时间。

练习2：实现 Best-Fit 连续物理内存分配算法（需要编程）

在完成练习一后，参考kern/mm/default_pmm.c对First Fit算法的实现，编程实现Best Fit页面分配算法，算法的时空复杂度不做要求，能通过测试即可。

First Fit和Best Fit都是内存分配算法，用于满足进程的内存分配请求，但它们在内存块选择上有一些显著的不同之处：

First Fit：

原理：First Fit算法从内存空闲块链表的起始位置开始查找，找到第一个足够大的内存块来满足进程的分配请求。

速度：由于从链表的起始位置开始查找，First Fit通常更快地找到可用的内存块，因此在分配速度上具有优势。

内存碎片：First Fit可能导致内存碎片，因为它会选择第一个满足大小条件的块，而不是尝试最小化碎片。

适用性：First Fit通常用于需要快速分配内存的场景，但可以容忍一些内存碎片的应用。

Best Fit：

原理：Best Fit算法从内存空闲块链表中找到一个大小最接近进程需求的内存块。它会搜索整个链表以找到最佳匹配的块。

速度：由于需要搜索整个链表，Best Fit通常需要更多的时间来找到适合的块，因此在分配速度上可能不如First Fit。

内存碎片：Best Fit在某种程度上可以减少内存碎片，因为它选择最接近需求大小的块。然而，它仍然可能导致一些内存碎片。

适用性：Best Fit通常用于需要最小化内存碎片的应用，但可以容忍较慢的分配速度。

best_fit_init_memmap:

```
static void
best_fit_init_memmap(struct Page *base, size_t n) {
    assert(n > 0);
    struct Page *p = base;
    for (; p != base + n; p++) {
        assert(PageReserved(p));

        /*LAB2 EXERCISE 2: 2111454*/
        // 清空当前页框的标志和属性信息，并将页框的引用计数设置为0
        // 结构体Page的相关定义见memlayout.h
        p->flags = 0;
        p->property = 0;
        set_page_ref(p, 0);
    }
    base->property = n;
    SetPageProperty(base);
    nr_free += n;
    if (list_empty(&free_list)) {
        list_add(&free_list, &(base->page_link));
    } else {
        list_entry_t* le = &free_list;
        while ((le = list_next(le)) != &free_list) {
            struct Page* page = le2page(le, page_link);
            /*LAB2 EXERCISE 2: 2111454*/
            // 编写代码
            // 1、当base < page时，找到第一个大于base的页，将base插入到它前面，并退出循环
            if (base < page)
            {
                list_add_before(&(page->page_link), &(base->page_link));
                break;
            }
            // 2、当list_next(le) == &free_list时，若已经到达链表结尾，将base插入到链表尾部
            if (list_next(le) == &free_list)
            {
                list_add_after(&(page->page_link), &(base->page_link));
            }
        }
    }
}
```

```

    }
}

```

best_fit_alloc_pages:

```

static struct Page *
best_fit_alloc_pages(size_t n) {
    assert(n > 0);
    if (n > nr_free) {
        return NULL;
    }
    struct Page *page = NULL;
    list_entry_t *le = &free_list;
    size_t min_size = nr_free + 1;
    /*LAB2 EXERCISE 2: 2111454*/
    // 下面的代码是first-fit的部分代码，请修改下面的代码改为best-fit
    // 遍历空闲链表，查找满足需求的空闲页框
    // 如果找到满足需求的页面，记录该页面以及当前找到的最小连续空闲页框数量
    while ((le = list_next(le)) != &free_list) {
        struct Page *p = le2page(le, page_link);
        if (p->property >= n && min_size > p->property) {
            page = p;
            min_size = p->property;
        }
    }

    if (page != NULL) {
        list_entry_t* prev = list_prev(&(page->page_link));
        list_del(&(page->page_link));
        if (page->property > n) {
            struct Page *p = page + n;
            p->property = page->property - n;
            SetPageProperty(p);
            list_add(prev, &(p->page_link));
        }
        nr_free -= n;
        ClearPageProperty(page);
    }
    return page;
}

```

best_fit_free_pages:

```

static void
best_fit_free_pages(struct Page *base, size_t n) {
    assert(n > 0);
    struct Page *p = base;
    for (; p != base + n; p++) {
        assert(!PageReserved(p) && !PageProperty(p));
    }
}

```

```

        p->flags = 0;
        set_page_ref(p, 0);
    }
    /*LAB2 EXERCISE 2: 2111454*/
    // 编写代码
    // 具体来说就是设置当前页块的属性为释放的页块数、并将当前页块标记为已分配状态、最后
    增加nr_free的值
    base->property = n;
    SetPageProperty(base);
    nr_free += n;

    if (list_empty(&free_list)) {
        list_add(&free_list, &(base->page_link));
    } else {
        list_entry_t* le = &free_list;
        while ((le = list_next(le)) != &free_list) {
            struct Page* page = le2page(le, page_link);
            if (base < page) {
                list_add_before(le, &(base->page_link));
                break;
            } else if (list_next(le) == &free_list) {
                list_add(le, &(base->page_link));
            }
        }
    }

    list_entry_t* le = list_prev(&(base->page_link));
    if (le != &free_list) {
        p = le2page(le, page_link);
        /*LAB2 EXERCISE 2: 2111454*/
        // 编写代码
        // 1、判断前面的空闲页块是否与当前页块是连续的，如果是连续的，则将当前页块合并
        到前面的空闲页块中
        // 2、首先更新前一个空闲页块的大小，加上当前页块的大小
        // 3、清除当前页块的属性标记，表示不再是空闲页块
        // 4、从链表中删除当前页块
        // 5、将指针指向前一个空闲页块，以便继续检查合并后的连续空闲页块
        if (p + p->property == base)
        {
            p->property += base->property;
            ClearPageProperty(base);
            list_del(&(base->page_link));
            base = p;
        }
    }

    le = list_next(&(base->page_link));
    if (le != &free_list) {
        p = le2page(le, page_link);
        if (base + base->property == p) {
            base->property += p->property;
            ClearPageProperty(p);
            list_del(&(p->page_link));
        }
    }

```


} }

请在实验报告中简要说明你的设计实现过程，阐述代码是如何对物理内存进行分配和释放，并回答如下问题：

- 你的 Best-Fit 算法是否有进一步的改进空间?

[illegible]

扩展练习Challenge: buddy system (伙伴系统) 分配算法 (需要编程)

Buddy System算法把系统中的可用存储空间划分为存储块(Block)来进行管理, 每个存储块的大小必须是2的n次幂($Pow(2, n)$), 即1, 2, 4, 8, 16, 32, 64, 128...

- 参考[伙伴分配器的一个极简实现](#)，在ucore中实现buddy system分配算法，要求有比较充分的测试用例说明实现的正确性，需要有设计文档。

扩展练习Challenge：任意大小的内存单元slub分配算法（需要编程）

slub算法，实现两层架构的高效内存单元分配，第一层是基于页大小的内存分配，第二层是在第一层基础上实现基于任意大小的内存分配。可简化实现，能够体现其主体思想即可。

- 参考[linux的slub分配算法/](#)，在ucore中实现slub分配算法。要求有比较充分的测试用例说明实现的正确性，需要有设计文档。

扩展练习Challenge：硬件的可用物理内存范围的获取方法（思考题）

- 如果 OS 无法提前知道当前硬件的可用物理内存范围，请问你有何办法让 OS 获取可用物理内存范围？

Challenges是选做，完成Challenge的同学可单独提交Challenge。完成得好的同学可获得最终考试成绩的加分。